

Figure 4: Output GUI of demo3.py

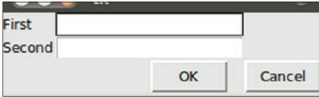


Figure 5: Output dialogue of demo4.py

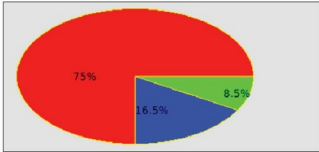


Figure 6: Output of the demo5.py

```
self.e1.focus_set() #5
self.e2=Entry(self.parent)
self.e2.grid(row=1, column=1)

self.b1=Button(self.parent, borderwidth=2, text="OK",
width=5) #6
self.b1.grid(row=2, column=1, columnspan=2)
self.b1.bind("<ButtonPress-1>", self.ok)
self.b2=Button(self.parent, borderwidth=2, text="Cancel",
width=5)
self.b2.grid(row=2, column=2, sticky=W)
self.b2.bind("<ButtonPress-1>", self.quit)

def ok(self, event=None): #7
    print "value is : ", self.e1.get(), self.e2.get()
    self.parent.destroy()

def quit(self, event=None):
    self.parent.destroy()

root=Tk()
d=diag(root)
root.mainloop()
```

Let's try to understand the program, with reference to the numbered lines:

1. We defined the constructor of the class 'diag', in which root is the parent widget.
2. We called the bind function to handle an event. When <return> was hit, it called the ok function; for <escape>, it called the quit function (defined in the class).

3. We have defined the place of the widget label by assigning it Row number 0. You can also assign a column to the widget according to your need.
4. We used an Entry class to let the user enter text.
5. This set the focus to the Entry widget (making it the active control) when the program is run.
6. We used the Button class to create buttons.
7. And defined the ok and quit functions to handle the respective events.

It's so good to program using Tkinter. You define every single step according to your need, and can play with the widget in your own way. One more piece of good news is that you can use the most popular canvas widget too, by using the Canvas class—it provides structured graphical facilities for Tkinter, and is mostly for drawings like graphs, etc. Here's a very small demo of how to make a stats analysis using the Canvas class (demo5.py; output shown in Figure 6):

```
from Tkinter import *
root=Tk()
c=Canvas(root)
c.pack()
xy=20, 20, 300, 180 #create an arc enclosed by the given
rectangle
c.create_arc(xy, start=0, extent=270, fill="red",
outline="yellow")
c.create_text(100, 100, text="75%")
c.create_arc(xy, start=270, extent=60, fill="blue",
outline="yellow")
c.create_text(180, 140, text="16.5%")
c.create_arc(xy, start=330, extent=30, fill="green",
outline="yellow")
c.create_text(280, 120, text="8.5%")
root.mainloop()
```

In this program, xy are the end-points of the rectangle in which the arcs are to be enclosed. Then we called the Canvas class, and using that widget, we created arcs with appropriate text positions. For more details regarding Canvas, refer to the Web.

I hope you liked playing around with GUIs using the Tkinter tool-kit. You can also combine the use of both PyZenity and Tkinter in the same program. Queries and suggestions are always welcome, so feel free to ping me! 

By: Ankur Aggarwal

The author loves to explore open source technologies and enjoys programming. He has learned C, C++, core Java, and Python. He loves rock/metal music and has been trying his hand at the guitar. He has also started a blog recently: www.flossstuff.wordpress.com and can be contacted via coolankur2006@gmail.com or on Twitter as @ankurtwi.